

OrbitWatch

Sistema de Monitoramento de Detritos Espaciais

v1.0 | Documentação Técnica

Linguagem	Python 3.12
Bibliotecas	poliastro, astropy, numpy, matplotlib, sgp4, requests
Fonte de dados	Space-Track.org (18th Space Defense Squadron / USAF)
Arquitetura	Modular — múltiplos arquivos por responsabilidade

1. Visão Geral do Projeto

O OrbitWatch é um sistema de monitoramento de detritos espaciais e satélites ativos desenvolvido em Python, projetado para auxiliar no acompanhamento de objetos em órbita terrestre. O sistema permite o cadastro manual de corpos celestes com suas respectivas posições e velocidades, além da importação de dados reais diretamente da API do Space-Track.org, base de dados oficial operada pela Força Aérea dos Estados Unidos. Com os objetos cadastrados, o usuário pode calcular a distância euclidiana tridimensional entre dois corpos, simular suas trajetórias orbitais ao longo do tempo e receber alertas automáticos quando a distância entre eles cai abaixo de um limiar configurável — indicando risco de colisão. Ao término de cada simulação, o sistema gera automaticamente um relatório visual em formato PNG contendo o gráfico de distância ao longo do tempo e um resumo dos dados analisados. Toda a arquitetura do projeto foi desenvolvida de forma modular, com cada funcionalidade isolada em seu próprio arquivo, facilitando a manutenção, a evolução do código e o trabalho em equipe.

Estrutura de Arquivos

Arquivo / Pasta	Responsabilidade
main.py	Ponto de entrada — menu principal
modulos/cadastro.py	Cadastro manual de satélites e detritos
modulos/listagem.py	Listagem paginada de objetos cadastrados
modulos/distancia.py	Cálculo de distância entre dois corpos
modulos/simulacao.py	Simulação orbital e alertas de colisão
modulos/deletar.py	Remoção de objetos cadastrados
modulos/tle.py	Integração com API Space-Track (dados reais)
dados/objetos.json	Banco de dados local dos objetos cadastrados
relatorios/	Gráficos PNG gerados pelas simulações

2. Menu Principal — main.py

O main.py é o ponto de entrada do sistema. Ele não contém lógica própria — apenas orquestra os módulos através de um menu interativo. Utiliza um dicionário de opções para mapear cada entrada do usuário a uma função executar() do módulo correspondente.

Opcao	Funcionalidade	Modulo
1	Descrição do projeto	main.py (inline)
2	Cadastrar satélite/detrito	modulos/cadastro.py
3	Listar objetos cadastrados	modulos/listagem.py
4	Calcular distancia entre corpos	modulos/distancia.py
5	Simulação de trajetória	modulos/simulacao.py
6	Deletar objeto	modulos/deletar.py
7	Importar TLE (Space-Track)	modulos/tle.py
0	Sair do sistema	—

➔ Menu em operação:

```
1 -- Descrição do projeto
2 -- Cadastrar satélite ativo / detrito
3 -- Cadastrar TLE
4 -- Deletar objeto
5 -- Listar objetos
6 -- Calcular distância entre corpos
7 -- Simulação de trajeto
0 -- Sair

0 que deseja realizar? ---
```

3. Módulos do Sistema

3.1 Cadastro — modulos/cadastro.py

Responsável pelo cadastro manual de satélites ativos e detritos espaciais. Coleta nome, tipo, posição (X, Y, Z em km) e velocidade (X, Y, Z em km/s) via input. Valida automaticamente se a velocidade ultrapassa a velocidade de escape da Terra (11,2 km/s), alertando o usuário sobre possíveis inconsistências na simulação. Os dados são persistidos em dados/objetos.json com encoding UTF-8.

Funcao	Descricao
<code>salvar_objeto(corpo)</code>	Salva um dicionário de objeto no JSON. Reutilizada pelo modulo TLE.
<code>cadastro_inicial()</code>	Coleta e valida todos os dados do usuário. Retorna (nome, tipo, posição, velocidade).
<code>executar()</code>	Função principal chamada pelo menu. Orquestra coleta e salvamento.

→ Cadastro manual em operação:

```

1 -- Descrição do projeto
2 -- Cadastrar satélite ativo / detrito
3 -- Cadastrar TLE
4 -- Deletar objeto
5 -- Listar objetos
6 -- Calcular distância entre corpos
7 -- Simulação de trajeto
0 -- Sair

O que deseja realizar? --- 2
Nome: Demonstração
Tipo (satélite ou detrito): satélite
Posição X (km): 5340
Posição Y (km): 1040
Posição Z (km): 3349
Velocidade X (km/s): 1.4
Velocidade Y (km/s): 2.3
Velocidade Z (km/s): 5.1
✅ Objeto cadastrado com sucesso!

```

```

O que deseja realizar? --- 2
Nome: demonstração 2
Tipo (satélite ou detrito): detrito
Posição X (km): 210
Posição Y (km): 2330
Posição Z (km): 4000
Velocidade X (km/s): 14
Velocidade Y (km/s): 2
Velocidade Z (km/s): 4
⚠️ Atenção: velocidade de 14.70 km/s excede a velocidade de escape da Terra (11.2 km/s)!
Este objeto não estará em órbita terrestre e pode causar erros na simulação.
Deseja continuar ou refazer cadastro? (C/RC)RC

```

3.2 TLE / Space-Track — `modulos/tle.py`

Integração com a API oficial do Space-Track.org para importação de dados reais de objetos em órbita. Gerencia sessão autenticada com reuso (login único por execução). Converte dados TLE (Two-Line Elements) em vetores de posição e velocidade usando a biblioteca `sgp4`, compatíveis com o formato do sistema.

Funcao	Descricao
<code>obter_sessao()</code>	Retorna sessão ativa ou realiza login. Usa variável global <code>_sessao</code> para reuso.
<code>buscar_tle(session, nome)</code>	Busca objetos na API por nome (suporta coringa <code>~~</code>). Retorna lista de resultados JSON.
<code>tle_para_orbita(dados)</code>	Converte TLE em dicionário de objeto usando <code>sgp4</code> . Calcula posição/velocidade para o momento atual.
<code>executar()</code>	Fluxo completo: login, busca, exibição e salvamento opcional no <code>objetos.json</code> .

→ Cadastro TLE em operação:

```
Nome do objeto (ex: ISS, HUBBLE): AQUA
Encontrado: SAC-D (AQUARIUS)
TLE Line 1: 1 37673U 11024A 26159.51397676 .00000923 00000-0 13320-3 0 9997
TLE Line 2: 2 37673 97.9765 173.5090 0001872 104.6099 14.1544 14.78618632806507

Nome: SAC-D (AQUARIUS)
Posição: X=3826.26 Y=-1227.55 Z=-5755.85 km
Velocidade: X=-6.2582 Y=0.0908 Z=-4.1825 km/s
Deseja salvar esses dados? (s/n)s
✅ Objeto cadastrado com sucesso!
```

3.3 Exclusão de objeto— modulos/deletar.py

O módulo deletar.py é responsável pela exclusão de objetos cadastrados no sistema. Ao ser executado, carrega e exibe a lista completa de objetos armazenados, solicita que o usuário selecione o item desejado pelo número correspondente e pede uma confirmação antes de efetuar a remoção. Após confirmada, a lista atualizada é salva automaticamente no arquivo objetos.json, garantindo a persistência da alteração.

Funcao	Descricao
<code>executar()</code>	Carrega objetos, exibe lista, solicita escolha e confirmação, remove e salva.

→ Exclusão em operação:

```
[9] Hostil – Detrito
Posição: X= 450.00 Y= 2000.00 Z= 3500.00 km
Velocidade: X= 2.4000 Y= 3.1000 Z= 0.1000 km/s

[10] Pog Ger – Satélite
Posição: X= 6.00 Y= 7.00 Z= 69.00 km
Velocidade: X= 4.0000 Y= 2.0000 Z= 0.0000 km/s

[11] SAC-D (AQUARIUS) – satélite
Posição: X= 3826.26 Y= -1227.55 Z= -5755.85 km
Velocidade: X= -6.2582 Y= 0.0908 Z= -4.1825 km/s

[12] Demonstração – Satélite
Posição: X= 5340.00 Y= 1040.00 Z= 3349.00 km
Velocidade: X= 1.4000 Y= 2.3000 Z= 5.1000 km/s

Objeto a deletar: 3

Tem certeza que deseja remover Quandale? (s/n) --> s

Removendo Quandale...
Objeto removido com sucesso!
```

3.4 Listagem — `modulos/listagem.py`

Exibe todos os objetos cadastrados com paginação de 10 itens por vez. Mostra posição com 2 casas decimais e velocidade com 4, mantendo a precisão total no JSON. Exporta funções reutilizáveis por outros módulos.

Funcao	Descricao
<code>carregar_objetos()</code>	Le e retorna a lista completa do objetos.json.
<code>exibir_objetos(objetos, inicio=0)</code>	Printa os objetos formatados. O parâmetro início garante numeração continua entre páginas.
<code>executar()</code>	Loop de paginação com navegação s/n pelo usuário.

→ Listagem em operação:

```
O que deseja realizar? --- 5

[1] Dot - satélite
Posição:   X= 6000.00 Y= 2000.00 Z= 2500.00 km
Velocidade: X= 0.1000 Y= 7.0000 Z= 2.0000 km/s

[2] teste - satélite
Posição:   X= 200.00 Y= 1000.00 Z= 6300.00 km
Velocidade: X= 2.0000 Y= 1.0000 Z= 4.0000 km/s

[3] Quandale - Detrito
Posição:   X= 200.00 Y= 4000.00 Z= 10.00 km
Velocidade: X= 0.0000 Y= 15.0000 Z= 0.0000 km/s

[4] Dingle - Satélite
Posição:   X= 2000.00 Y= 1000.00 Z= 500.00 km
Velocidade: X= 2.0000 Y= 1.5000 Z= 1.0000 km/s

[5] AISSAT 1 - satélite
Posição:   X= -4376.42 Y= -5220.06 Z= -1415.50 km
Velocidade: X= 0.1368 Y= 1.8563 Z= -7.3375 km/s

[6] LEMUR 2 HUBBLE-5 - satélite
Posição:   X= 218.99 Y= -6089.97 Z= -3378.62 km
Velocidade: X= -1.0380 Y= -3.6547 Z= 6.5383 km/s

[7] SKYTERRA 1 - satélite
Posição:   X= -38296.01 Y= 17266.68 Z= 3619.73 km
Velocidade: X= -1.2699 Y= -2.7993 Z= -0.0709 km/s

[8] NOAA 19 - satélite
Posição:   X= 3510.66 Y= 2066.59 Z= -5988.59 km
Velocidade: X= -4.0292 Y= -4.7502 Z= -4.0070 km/s

[9] Hostil - Detrito
Posição:   X= 450.00 Y= 2000.00 Z= 3500.00 km
Velocidade: X= 2.4000 Y= 3.1000 Z= 0.1000 km/s

[10] Pog Ger - Satélite
Posição:   X= 6.00 Y= 7.00 Z= 69.00 km
Velocidade: X= 4.0000 Y= 2.0000 Z= 0.0000 km/s
Ver mais? (s/n) -- s

[11] SAC-D (AQUARIUS) - satélite
Posição:   X= 3826.26 Y= -1227.55 Z= -5755.85 km
Velocidade: X= -6.2582 Y= 0.0908 Z= -4.1825 km/s

[12] Demonstração - Satélite
Posição:   X= 5340.00 Y= 1040.00 Z= 3349.00 km
Velocidade: X= 1.4000 Y= 2.3000 Z= 5.1000 km/s

Fim da lista!
```

3.5 Distância — `modulos/distancia.py`

Calcula a distância euclidiana 3D momentânea entre dois objetos cadastrados usando a formula de Pitágoras generalizada para três dimensões, implementada via `numpy.linalg.norm`. O usuário seleciona os objetos pelo número da lista.

Funcao	Descricao
<code>escolha_objeto(mensagem, dados)</code>	Valida e retorna um objeto da lista pelo número escolhido. Reutilizada por <code>simulacao.py</code> e <code>deletar.py</code> .
<code>executar()</code>	Carrega objetos, solicita escolha dos dois corpos e exibe a distância em km.

→ Distância e operação:

- Alerta: É importante mencionar que tanto no sistema de calculo de distância, deleção e simulação são listados todos os objetos cadastrados para melhor visualização e utilização de dados

```
[10] Pog Ger - Satélite
Posição: X= 6.00 Y= 7.00 Z= 69.00 km
Velocidade: X= 4.0000 Y= 2.0000 Z= 0.0000 km/s

[11] SAC-D (AQUARIUS) - satélite
Posição: X= 3826.26 Y= -1227.55 Z= -5755.85 km
Velocidade: X= -6.2582 Y= 0.0908 Z= -4.1825 km/s

[12] Demonstração - Satélite
Posição: X= 5340.00 Y= 1040.00 Z= 3349.00 km
Velocidade: X= 1.4000 Y= 2.3000 Z= 5.1000 km/s

Para o cálculo de distância, deverá ser utilizados apenas dois corpos já cadastrados!

Corpo 1 --> 12
Corpo 2 --> 11

Distância entre Demonstração e SAC-D (AQUARIUS): 9504.29 km

Fazer outra conta? (s/n) --
```

3.6 Simulação — `modulos/simulacao.py`

Modulo central do sistema. Propaga as orbitas de dois objetos no tempo usando a biblioteca `poliastro`, calculando a distância entre eles a cada intervalo definido pelo usuário. Dispara alertas quando a distância cai abaixo do limiar configurado. Ao final, gera um gráfico PNG com a curva de distância ao longo do tempo, linha de limiar de alerta e rodapé com informações da simulação.

Funcao	Descricao
<code>calc_orbita(corpo)</code>	Converte um dicionário de objeto em um objeto <code>Orbit</code> da <code>poliastro</code> .
<code>tempo()</code>	Solicita e valida tempo total (horas) e intervalo (minutos). Retorna (minutos_total, intervalo).
<code>executar()</code>	Orquestra toda a simulação: escolha de objetos, propagação, alertas, gráfico e salvamento.

→ Simulação em operação:

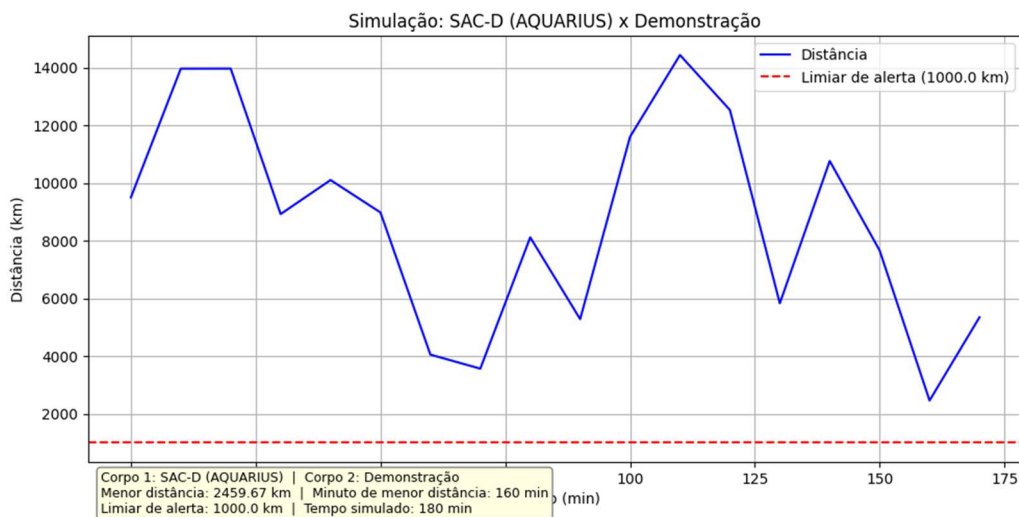
```
[11] SAC-D (AQUARIUS) - satélite
Posição: X= 3826.26 Y= -1227.55 Z= -5755.85 km
Velocidade: X= -6.2582 Y= 0.0908 Z= -4.1825 km/s

[12] Demonstração - Satélite
Posição: X= 5340.00 Y= 1040.00 Z= 3349.00 km
Velocidade: X= 1.4000 Y= 2.3000 Z= 5.1000 km/s

Para a simulação de órbita, deverá ser utilizados apenas dois corpos já cadastrados!

Corpo 1 --> 11
Corpo 2 --> 12
Tempo de simulação (horas) --> 3
Intervalo de verificação (minutos) -->
Digite apenas números!
Tempo de simulação (horas) --> 3
Intervalo de verificação (minutos) --> 10
Limiar de alerta (km) --> 1000
Simulação finalizada
Menor distância detectada: 2459.67 km
minuto: 160
Deseja salvar o gráfico para relatório? (s/n): s
✓ Relatório salvo em relatorios/relatorio_08062026_163246.png
```

→ Gráfico gerado:



4. Fluxo de Dados

Todos os objetos do sistema — cadastrados manualmente ou importados via TLE — são armazenados no arquivo `dados/objetos.json` com a seguinte estrutura padrão:

```
{ "nome": "ISS", "tipo": "satélite", "posicao": [x, y, z], "velocidade": [vx, vy, vz] }
```

Os módulos de listagem, distancia, simulação e deletar sempre leem desse arquivo como fonte única de dados. Isso garante que objetos reais (TLE) e fictícios coexistam no mesmo banco e possam ser comparados e simulados juntos.

5. Dependências — requirements.txt

Biblioteca	Versao	Uso no projeto
numpy	latest	Cálculos vetoriais e distancia euclidiana 3D
matplotlib	latest	Geração de gráficos PNG das simulações
astropy	latest	Unidades físicas (km, km/s) e tempo astronômico
poliastro	latest	Propagação orbital e criação de orbitas
requests	latest	Requisições HTTP para API do Space-Track
sgp4	latest	Conversao de TLE em posição e velocidade

6. Segurança e Boas Práticas

Dados locais no .gitignore

O arquivo dados/objetos.json e a pasta relatorios/ não são versionados, pois contém dados gerados localmente por cada usuário.

Validação de velocidade

O modulo de cadastro calcula a magnitude do vetor velocidade e alerta quando ultrapassa 11,2 km/s (velocidade de escape terrestre), evitando objetos fisicamente inválidos.

Tratamento de erros

Todos os inputs numéricos usam try/except ValueError. A API do Space-Track verifica o status HTTP antes de processar a resposta.

Reuso de sessão

A sessão autenticada com o Space-Track e armazenada em variável global (_sessao) e reutilizada, evitando logins repetidos e respeitando os limites de requisição da API.

7. Integrantes

- João Victor Pereira Gimenes - 571662
- Juan Duarte - 570331
- Julio Cesar da Silva Cocco - 569463
- Rafael Boletini de Oliveira - 570219
- Victor Rossi Sales Zanandre - 573844